Lab Overview
This hands on personal lab was designed to deepen my understanding of the OSI model, network protocols, and packet inspection (DPI). Using Wireshark on macOS, I captured, filtered, and analyzed live local network traffic to investigate standard network infrastructure processes and isolate common application layer security vulnerabilities.

Tools & Technologies Used
• Network Protocol Analyzer: Wireshark 4.6
• Interface: Wi-Fi
• Environment: Local Subnet (x.x.x.x/xx)

Lab Tasks & Technical Walkthrough
Task 1: Analyzing the Core Network Infrastructure (DHCP DORA Process)
• **Objective:** Isolate and observe how a client machine requests and secures a dynamic IP address from a DHCP server.
• **Methodology:** I opened the macOS Terminal and initiated a live packet capture on the wifi interface. To force a new lease negotiation, I forced a DHCP renewal via the command line: sudo ipconfig set xx DHCP
• **Wireshark Filter Applied**: dhcp
• **Analysis & Findings:** I successfully isolated the complete four step DORA sequence:
1. Discover: Broadcast packet sent by the Mac searching for an available DHCP server.
2. Offer: Unicast/Broadcast from the home gateway router offering an IP configuration.
3. Request: Client response accepting the configuration.
4. Acknowledge: Server confirmation finalizing the lease.
Inside the Bootstrap Protocol layer, I inspected the specific transaction IDs and verified the leased IP assignment parameters.

Task 2: Reconstructing an Unencrypted Authentication Stream
• **Objective:** Identify security risks associated with cleartext protocols by capturing and extracting sensitive payload data.
• **Methodology:** I navigated to an intentionally unencrypted testing web application (http://testfire.net) and submitted a dummy username and password string while capturing network streams.
• **Wireshark Filter Applied:** http.request.method == "POST"
• **Analysis & Findings:** By filtering exclusively for outbound HTTP POST data, I located the exact packet carrying the form submission. I utilized Wireshark's Follow -> HTTP Stream feature to reconstruct the conversation. Because the site lacked TLS encryption, the raw credentials (uid=Ican &passw=icanseethepassword) were completely readable in cleartext inside the application layer data payload.

Task 3: Evaluating Local Network Overhead & Noise Baseline
• **Objective:** Map out background subnet chatter and analyze protocol traffic distribution on a standard local area network.
• **Methodology:** I kept my host machine idle for 10 minutes to clear intentional browsing traffic, capturing background infrastructure packets. I then audited the results using Statistics -> Protocol Hierarchy.
• **Wireshark Filter Applied:** arp || mdns || llmnr
• **Analysis & Findings:** The protocol distribution network baseline revealed significant conversational overhead stemming from:
• **ARP** (Address Resolution Protocol): Constant broadcast inquiries mapping Layer 2 MAC addresses to Layer 3 IP addresses across local smart home devices.
• **mDNS (Multicast DNS):** Continuous traffic targeting 224.0.0.251 from Apple devices running AirPlay/Bonjour service discoveries.

Key Takeaways

1**. Encapsulation Visibility:** Gained practical visualization of how data wraps down through the OSI model layers from the Application layer data payload down to the physical Ethernet Frame headers.
2. Security Awareness: Verified why secure, encrypted transport protocols (HTTPS, SSH) are non negotiable for enterprise networks to mitigate credential sniffing threats.
3. Advanced Filtering Literacy: Developed a working knowledge of specialized display and capture syntax to quickly isolate target streams in high traffic production networks.